CLOUD & PLATFORMS

SAP AI Core

Run and serve ML on managed Kubernetes and reach governed foundation models through the Generative AI Hub and orchestration.

SAP AI Core · AI Launchpad · Generative AI Hub · orchestration · KServe · foundation-models · SAP BTP

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What SAP AI Core Is and Why It Matters
2. 2. Where It Fits in the SAP Landscape (When to Use It)
3. 3. Getting Started: Provisioning and Authentication
4. 4. Core Concepts and the Entity Lifecycle
5. 5. SAP AI Launchpad: The Operations Cockpit
6. 6. Running and Serving Your Own ML Workflows
7. 7. Generative AI Hub: Connecting Foundation Models
8. 8. Orchestration: A Harmonised, Governed Inference Pipeline
9. 9. Cheat-Sheet & Comparison Tables
10. 10. Best Practices
11. 11. Common Pitfalls
12. 12. Interview Questions & Answers
13. 13. Further Learning

SAP AI Core is the engine room for AI inside the SAP Business Technology Platform (BTP). It runs your training and inference workloads on Kubernetes, serves models behind stable endpoints, and — through the **Generative AI Hub** and the **Orchestration** service — gives you one governed, auditable door to foundation models from OpenAI, Anthropic, Google, Mistral, Meta and SAP itself. If your organisation lives in S/4HANA, SuccessFactors or Ariba, AI Core is how you add AI without smuggling data out to a random vendor API.

1. What SAP AI Core Is and Why It Matters

SAP AI Core is a **runtime**, not a modelling tool. You do not click together neural networks in it; you hand it containerised workflows and it executes them — on demand, at scale, with isolation and lineage. It is built on open standards: `Argo Workflows` for training pipelines, `KServe` for model serving, and a GitOps model (via Argo CD) for declaring what should run. On top of that infrastructure, the `Generative AI Hub` adds a managed proxy to the world's foundation models.

GOVERNED ACCESS TO LLMS

One contract, one audit trail, EU/role-based data residency. No scattered API keys, no data leaving your BTP subaccount uncontrolled.

RUNS YOUR OWN ML

Bring a Docker image + a workflow template and AI Core trains and serves it on managed GPU/CPU — no cluster to babysit.

DEEP SAP INTEGRATION

First-class hooks into SAP HANA Cloud (vector engine), Datasphere, S/4HANA and the ABAP SDK for generative AI.

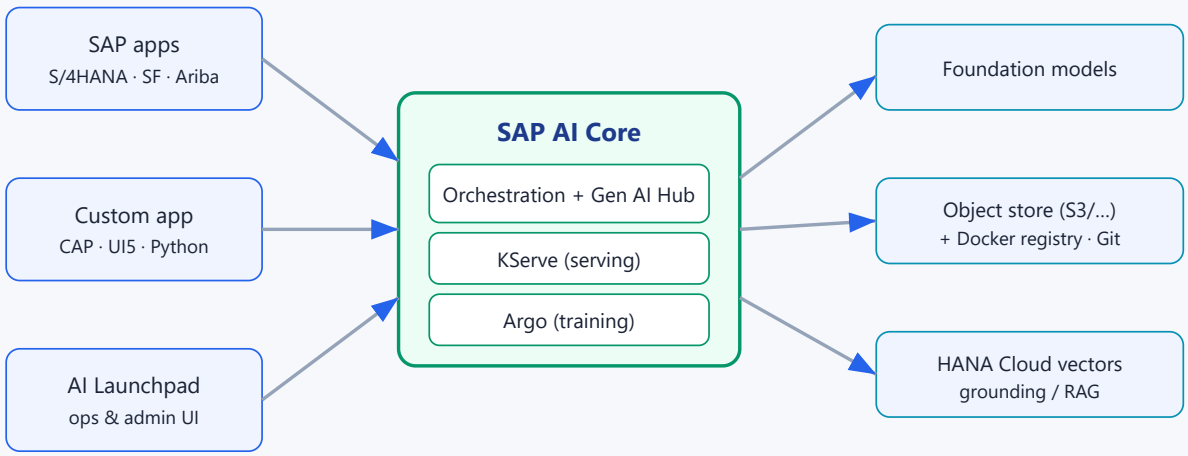


Fig 1 — AI Core sits between your SAP/custom apps and the AI resources (external LLMs, your object store, HANA vectors). Launchpad is the operations cockpit.

> **Mental model**
>
> Think of AI Core as a *managed Kubernetes for AI* plus a *gateway to foundation models*. The AI API (a REST API) is the universal remote control; AI Launchpad is a GUI over that same API; the SDKs are a Python wrapper over it.

2. Where It Fits in the SAP Landscape (When to Use It)

AI Core is one service in the broader BTP family. Knowing the neighbours tells you when to reach for it.

USE AI CORE WHEN...

You need governed access to LLMs for a custom app, you want to train/serve your own models on managed infra, or you are building RAG grounded in SAP data under enterprise compliance.

DON'T REACH FOR IT WHEN...

You just want a chatbot in a Fiori app with zero infra — SAP Joule or prebuilt AI business services may already do it. A pure prototype on your laptop doesn't need AI Core either.

Service	Role	Relationship to AI Core
SAP AI Core	Runtime for training/serving + LLM gateway	The subject of this guide
SAP AI Launchpad	Multi-tenant UI to operate AI runtimes	Front-end that calls AI Core's AI API
Generative AI Hub	Managed foundation-model access	A capability *inside* AI Core
SAP HANA Cloud	DB with native vector engine	Grounding/RAG store for orchestration
SAP BTP (CF/Kyma)	Platform & app runtime	Where AI Core is provisioned & consumed
SAP Joule	Copilot across SAP apps	Consumer of AI Core under the hood

3. Getting Started: Provisioning and Authentication

AI Core is consumed as a BTP service instance. You create an instance, generate a **service key**, and use its OAuth client credentials to get a bearer token for the AI API. The `extended` service plan unlocks the Generative AI Hub and real training/serving; the `free` /standard plan is for exploration.

BASH

```bash
# Via the Cloud Foundry CLI (one common path)
cf login -a https://api.cf.eu10.hana.ondemand.com
cf create-service aicore extended my-ai-core
cf create-service-key my-ai-core my-ai-core-key
cf service-key my-ai-core my-ai-core-key   # prints the JSON below
```

JSON

```json
{
  "clientid": "sb-xxxx!b123|aicore!b456",
  "clientsecret": "xxxxxxxx",
  "url": "https://xxxx.authentication.eu10.hana.ondemand.com",
  "ai_api_url": "https://api.ai.prod.eu-central-1.aws.ml.hana.ondemand.com",
  "identityzone": "mytenant",
  "serviceurls": { "AI_API_URL": "https://api.ai.prod.eu-central-1.aws.ml.hana.ondeman
}
```

BASH

```bash
# Install the Python SDKs
pip install ai-core-sdk            # low-level: workflows, deployments, admin
pip install ai-api-client-sdk      # the AI API client (often pulled in transitively)
pip install generative-ai-hub-sdk  # LLM access: OpenAI-compatible + LangChain
```

PYTHON

```python
# connect.py — authenticate and list scenarios
from ai_core_sdk.ai_core_v2_client import AICoreV2Client

client = AICoreV2Client(
    base_url="https://api.ai.prod.eu-central-1.aws.ml.hana.ondemand.com/v2",
    auth_url="https://xxxx.authentication.eu10.hana.ondemand.com/oauth/token",
    client_id="sb-xxxx!b123|aicore!b456",
    client_secret="xxxxxxxx",
    resource_group="default",   # pick your isolation unit (see §4)
)

for s in client.scenario.query().resources:
    print(s.id, "-", s.name)
```

✦ **Config over code**

The Gen AI Hub SDK auto-reads `AICORE_*` environment variables or `~/.aicore/config.json`, so production code rarely hard-codes credentials. Store the service key in a BTP destination or a secrets vault, never in Git.

4. Core Concepts and the Entity Lifecycle

Everything in AI Core is a REST resource. Internalise this vocabulary — interviews and docs assume it.

Entity	What it is
Resource group	A tenant/isolation boundary. Scenarios, configs, deployments and secrets live inside one. Great for separating teams, projects or environments.
Scenario	A logical use case (e.g. "demand forecasting"). A container for related executables and models.
Executable	A versioned *template* (Argo workflow or KServe serving template) stored in Git. Declares inputs, outputs and the container to run.
Configuration	An executable + concrete parameter values + input artifact bindings. The reusable "recipe" you instantiate.
Execution	A finished/running *training* job created from a (training) configuration. Produces output artifacts (models).
Deployment	A running *serving* instance created from a (serving) configuration. Exposes a stable inference URL.
Artifact	A registered pointer to data/model in the object store (dataset, model, resultset).
Resource plan	Compute size (e.g. `starter`, `infer.s`, `train.l`, GPU plans). Chosen per config.

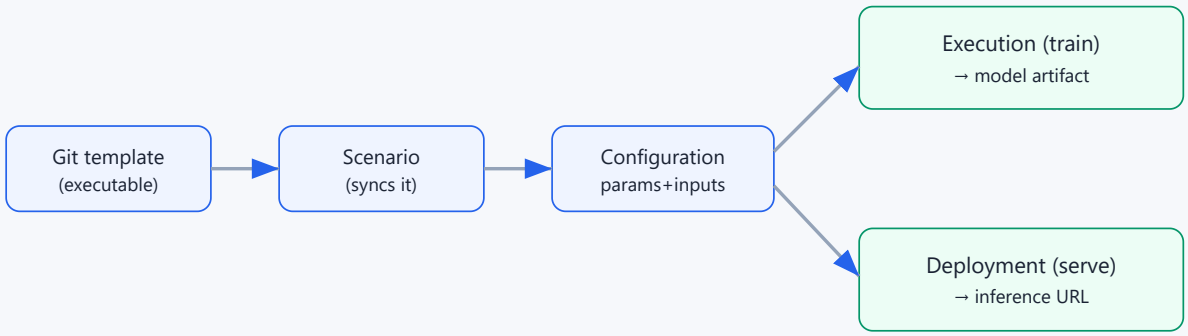


Fig 2 — Lifecycle: a Git template becomes a scenario's executable; a configuration binds parameters; an execution trains, a deployment serves.

5. SAP AI Launchpad: The Operations Cockpit

AI Launchpad is a separate BTP app and the human-friendly front end to the AI API. It is **multi-tenant**: one Launchpad can connect to several AI Core instances (dev, test, prod) via "connections".

Its apps map onto the entities above.

ML OPERATIONS

Browse scenarios, create configurations, launch executions, start/stop deployments, inspect logs and metrics.

GENERATIVE AI HUB

A chat playground + prompt editor to compare foundation models and build orchestration flows without code.

WORKSPACES & SAP AI API

Manage connections, resource groups, secrets (object store, Docker, Git) and scheduling.

> **Who uses what**
> Data scientists and ops engineers tend to live in Launchpad; application developers call the AI API/SDK directly from their services. Both hit the same backend.

6. Running and Serving Your Own ML Workflows

To run custom ML you provide three things: a **Docker image** (in a registry AI Core can read), a **workflow template** (in a Git repo AI Core syncs), and **secrets** that let it reach your object store, registry and Git. Training uses Argo Workflows; serving uses KServe.

Registering infrastructure (one-time)

PYTHON

```python
# Register a Git repo of templates, a Docker secret, and an object-store secret
client.repositories.create(
    name="my-templates",
    url="https://github.com/acme/ai-core-templates",
    username="acme-bot", password="ghp_xxx",
)
client.applications.create(
    application_name="demand-forecast",
    revision="HEAD", repository_url="https://github.com/acme/ai-core-templates",
    path="workflows",   # folder of YAML templates to sync as executables
)
client.docker_registry_secrets.create(
    name="docker-secret",
    data={".dockerconfigjson": "{ ... base64 auth ... }"},
)
```

A serving template (KServe)

YAML

```yaml
apiVersion: ai.sap.com/v1alpha1
kind: ServingTemplate
metadata:
  name: forecast-server
  annotations:
    scenarios.ai.sap.com/name: "demand-forecast"
    executables.ai.sap.com/name: "forecast-server"
  labels:
    ai.sap.com/version: "1.0.0"
spec:
  inputs:
    artifacts:
      - name: model          # bound at configuration time
  template:
    apiVersion: "serving.kserve.io/v1beta1"
    metadata:
      labels: { ai.sap.com/resourcePlan: "infer.s" }
    spec:
      predictor:
        imagePullSecrets: [ name: docker-secret ]
        containers:
          - name: kserve-container
            image: "docker.io/acme/forecast:1.0.0"
            ports: [ containerPort: 9001, protocol: TCP ]
            env:
              - name: STORAGE_URI
                value: "{{inputs.artifacts.model}}"
```

Create a deployment and call it

PYTHON

```python
# Build config → deployment, then invoke the model's inference endpoint
cfg = client.configuration.create(
    name="forecast-cfg",
    scenario_id="demand-forecast",
    executable_id="forecast-server",
    input_artifact_bindings=[{"key": "model", "artifact_id": MODEL_ARTIFACT_ID}],
)
dep = client.deployment.create(configuration_id=cfg.id)
# ... wait until dep.status == 'RUNNING', then AI Core exposes dep.deployment_url
import requests
resp = requests.post(
    f"{dep.deployment_url}/v2/predict",
    headers={**client.rest_client.get_token(), "AI-Resource-Group": "default"},
    json={"instances": [[12, 4, 2026]]},
)
print(resp.json())
```

> ⚠ **Deployments cost money while RUNNING**
>
> A serving deployment holds a compute slot and bills continuously until you `STOP` it — unlike a one-shot execution that ends. Always stop idle deployments in non-prod.

7. Generative AI Hub: Connecting Foundation Models

The Generative AI Hub is the fast path most teams actually use. You deploy a foundation model (which just provisions a proxy to the provider — no GPU of yours is used), get a stable deployment URL, and call it with an OpenAI-compatible API. Model availability evolves; as of 2026 the Hub spans GPT-4o/4.1 and o-series (OpenAI), Claude (Anthropic), Gemini (Google), Mistral, Meta Llama, and SAP embedding/translation models.

PYTHON

```python
# 1) Deploy a foundation model = create a config in the 'foundation-models' scenario
params = {"modelName": "gpt-4o", "modelVersion": "latest"}
cfg = client.configuration.create(
    name="gpt4o-cfg", scenario_id="foundation-models",
    executable_id="azure-openai",   # provider-specific executable
    parameter_bindings=[{"key": k, "value": v} for k, v in params.items()],
)
dep = client.deployment.create(configuration_id=cfg.id)
# dep.deployment_url is now your model endpoint
```

PYTHON

```python
# 2) The easy way: generative-ai-hub-sdk, OpenAI-compatible interface
from gen_ai_hub.proxy.native.openai import chat

response = chat.completions.create(
    model="gpt-4o",   # resolves to your deployed model in the Hub
    messages=[
        {"role": "system", "content": "You are a terse SAP assistant."},
        {"role": "user", "content": "Explain a resource group in one line."},
    ],
    temperature=0.2,
)
print(response.choices[0].message.content)
```

PYTHON

```python
# 3) LangChain integration — swap providers without touching your chain
from gen_ai_hub.proxy.langchain.init_models import init_llm
from langchain_core.prompts import ChatPromptTemplate

llm = init_llm("anthropic--claude-3.5-sonnet", temperature=0)
prompt = ChatPromptTemplate.from_template("Summarise this ticket: {text}")
chain = prompt | llm
print(chain.invoke({"text": "User cannot post invoice; dump in FB60."}).content)
```

PYTHON

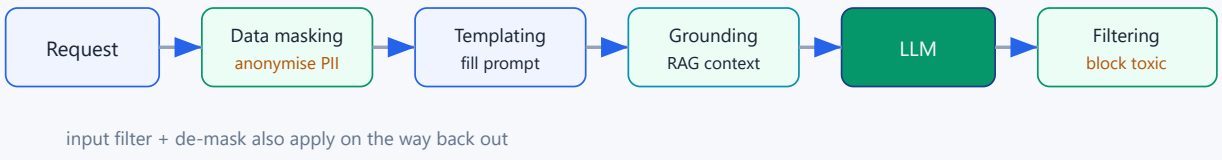
```python
# 4) Embeddings for RAG — same Hub, consistent governance
from gen_ai_hub.proxy.native.openai import embeddings
vec = embeddings.create(model="text-embedding-3-small", input="purchase order")
print(len(vec.data[0].embedding))   # e.g. 1536 dims
```

✦ **Why go through the Hub?**
Centralised billing, per-resource-group quotas, no provider keys sprawled across teams, EU data boundary options, and the ability to switch model vendors by changing one deployment — not your code.

8. Orchestration: A Harmonised, Governed Inference Pipeline

Calling a raw model is rarely enough in an enterprise. The **Orchestration** service wraps a single inference call in composable modules that run in a fixed order, all configured by one JSON/YAML object. This is the recommended way to consume LLMs in production.





input filter + de-mask also apply on the way back out

Fig 3 — The orchestration pipeline: one request flows through optional modules; swapping the model is a one-field change.

An orchestration config and call

PYTHON

```python
from gen_ai_hub.orchestration.models.config import OrchestrationConfig
from gen_ai_hub.orchestration.models.llm import LLM
from gen_ai_hub.orchestration.models.template import Template, TemplateValue
from gen_ai_hub.orchestration.models.message import SystemMessage, UserMessage
from gen_ai_hub.orchestration.models.content_filtering import (
    ContentFiltering, InputFiltering, OutputFiltering)
from gen_ai_hub.orchestration.models.azure_content_filter import AzureContentFilter
from gen_ai_hub.orchestration.service import OrchestrationService

template = Template(messages=[
    SystemMessage("You are a helpful SAP support agent."),
    UserMessage("Classify the sentiment of: {{?review}}"),
])
llm = LLM(name="gpt-4o", parameters={"temperature": 0.1})
cf = AzureContentFilter(hate=2, violence=2, sexual=2, self_harm=2)

config = OrchestrationConfig(
    template=template,
    llm=llm,
    filtering=ContentFiltering(input=InputFiltering(filters=[cf]),
                               output=OutputFiltering(filters=[cf])),
)

svc = OrchestrationService(api_url=ORCH_DEPLOYMENT_URL, config=config)
result = svc.run(template_values=[TemplateValue("review", "Shipment was 3 weeks late!"
print(result.orchestration_result.choices[0].message.content)
```

✓ **Grounding is built in**

Point the grounding module at a SAP HANA Cloud vector collection or the Document Grounding service and orchestration retrieves + injects context for you — a managed RAG without hand-wiring a retriever.

9. Cheat-Sheet & Comparison Tables

Task	How
Get a token	OAuth client-credentials against `url` + `/oauth/token`
Scope a call to a tenant	Header `AI-Resource-Group`
Sync templates from Git	Create an *Application* (GitOps)
Train a model	Configuration → Execution (Argo)
Serve a model	Configuration → Deployment (KServe)
Call an LLM (simple)	Gen AI Hub SDK, OpenAI-compatible
Call an LLM (governed)	Orchestration service
Stop billing	STOP the deployment / DELETE config

Dimension	Own ML workflows	Generative AI Hub
Compute	Your Docker image on AI Core GPU/CPU	Managed proxy to provider (no GPU of yours)
Primitive	Argo (train) / KServe (serve)	Model deployment + Orchestration
You supply	Image, template, data, secrets	Just a prompt/config
Typical user	Data scientist / ML engineer	App developer
Billing	Compute time (capacity units)	Tokens + deployment uptime

10. Best Practices

✓ **Isolate with resource groups**
One per team/environment. They cap quotas, scope secrets and keep one project's artifacts invisible to another — cheap, strong multi-tenancy.

✦ **Prefer orchestration over raw model calls**

You get masking, content filtering, grounding and provider-swap for free, and a single auditable contract. Reserve the native OpenAI interface for quick spikes.

✦ **GitOps everything**

Keep templates in Git and let AI Core sync them. Versions are reproducible, reviewable and roll back cleanly — never create executables by hand in prod.

✓ **Right-size resource plans**

Start on the smallest plan (`infer.s`) and scale up only on measured latency/throughput need. GPU plans are expensive.

11. Common Pitfalls

✕ **Forgetting AI-Resource-Group**

Omitting the header (or using the wrong group) returns empty lists or 404s on resources that clearly exist — the single most common "it's broken" moment.

⚠ **Leaving deployments RUNNING**

Idle serving deployments burn capacity units 24/7. Automate STOP for non-prod and monitor the metering dashboard.

⚠ **Hard-coding model names everywhere**

Model versions get deprecated. Centralise the model choice in an orchestration config so a retirement is a one-line change, not a code hunt.

✕ **Confusing executions and deployments**

Executions are finite training jobs that terminate; deployments are long-lived servers. Using the wrong template kind is a frequent beginner error.

⚠ **Shipping PII to the model unmasked**

Enable the data-masking module for anything touching personal data — compliance teams will ask, and it is trivial to turn on.

12. Interview Questions & Answers

What is SAP AI Core and how does it differ from AI Launchpad?

AI Core is the backend runtime (Kubernetes-based) that trains and serves models and gateways foundation models via the AI API. AI Launchpad is a BTP UI application that calls that same AI API to let humans operate scenarios, deployments and the Gen AI Hub. One is the engine; the other is the dashboard.

Explain the entity chain from template to prediction.

A Git template defines an *executable* synced into a *scenario*. A *configuration* binds parameters and input artifacts to that executable. From a training config you create an *execution* (Argo) that outputs a model artifact; from a serving config you create a *deployment* (KServe) that exposes an inference URL.

What problem does a resource group solve?

Multi-tenancy and isolation inside one AI Core instance. It scopes scenarios, configurations, deployments, artifacts and secrets, and lets you apply separate quotas — so teams or environments don't see or starve each other. You select it via the `AI-Resource-Group` header.

Why use the Generative AI Hub instead of calling OpenAI directly?

Governance: one contract and audit trail, centralised billing and quotas, no sprawl of provider keys, data-residency controls, and the ability to switch model vendors without touching application code. You also gain the orchestration modules (masking, filtering, grounding).

What does the orchestration service add over a raw model call?

A composable, server-side pipeline around one inference: templating, data masking (PII anonymisation), grounding (RAG), and input/output content filtering — all declared in one config. It harmonises providers behind a single API, so swapping GPT-4o for Claude is a one-field change.

Training uses Argo, serving uses KServe — why two?

They model different workloads. Training is a finite, possibly multi-step pipeline (Argo Workflows) that runs to completion and produces artifacts. Serving is a long-lived, autoscaling inference server (KServe) exposing an endpoint. Different lifecycles, different billing, different templates.

> **How does AI Core do RAG grounded in SAP data?**
>
> Embeddings via the Gen AI Hub feed a vector store — typically SAP HANA Cloud's native vector engine or the Document Grounding service. The orchestration grounding module retrieves relevant chunks at query time and injects them into the prompt, so the model answers from governed enterprise data.

> **What are capacity units / how is AI Core billed?**
>
> Compute is metered in capacity units tied to resource plans and uptime for executions and deployments; foundation-model usage adds token-based consumption. Idle running deployments still bill, so stopping them is a real cost lever.

> **How would you deploy a custom scikit-learn model on AI Core?**
>
> Containerise an inference server (e.g. a Flask/KServe predictor) into a Docker image; register the registry, object-store and Git secrets; commit a ServingTemplate; sync it as an executable via an Application; create a configuration binding the model artifact; then create a deployment and call its URL with the resource-group header.

> **When would you NOT use AI Core?**
>
> For out-of-the-box SAP AI (Joule, embedded AI business services) where no custom infra is needed, for throwaway laptop prototypes, or when a lightweight managed LLM API with no SAP-governance requirements suffices. AI Core shines when you need enterprise governance, SAP-data grounding, or to run your own models at scale.

> **How do you keep the deployment reproducible across dev/test/prod?**
>
> GitOps: templates live in Git and are synced via Applications, so each environment pins a revision. Combine with separate resource groups (or instances) per environment and configuration objects that externalise parameters — promotion becomes a Git revision bump, not manual clicking.

> **What is the AI-Resource-Group header and what breaks without it?**
>
> It routes every AI API call to a specific resource group. Omit it or use a wrong value and you get empty query results or 404s even when resources exist, because you are effectively looking in the wrong tenant partition.

13. Further Learning

- **SAP Help Portal — AI Core & AI Launchpad**: authoritative docs for the AI API, templates and the Gen AI Hub.

- **SAP Discovery Center / tutorials (developers.sap.com)**: hands-on missions provisioning AI Core and deploying a first model.
- **SDKs**: read the source/readmes for `ai-core-sdk`, `ai-api-client-sdk` and `generative-ai-hub-sdk`.
- **Argo Workflows & KServe docs**: the open-source primitives beneath AI Core — understanding them demystifies templates.
- **SAP HANA Cloud vector engine**: pair with orchestration grounding to build governed RAG.
- **Build a project**: wire a CAP/Node app → orchestration (masking + grounding) → HANA vectors for an end-to-end, interview-ready portfolio piece.